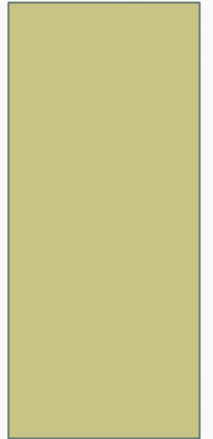


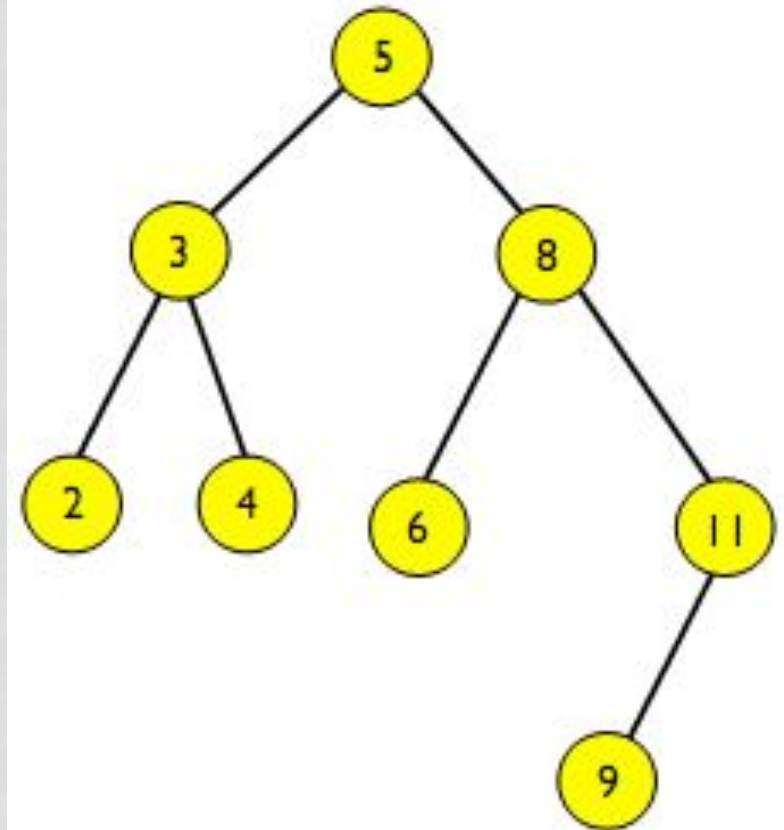
EC-200 DATA STRUCTURES

LECTURE 11

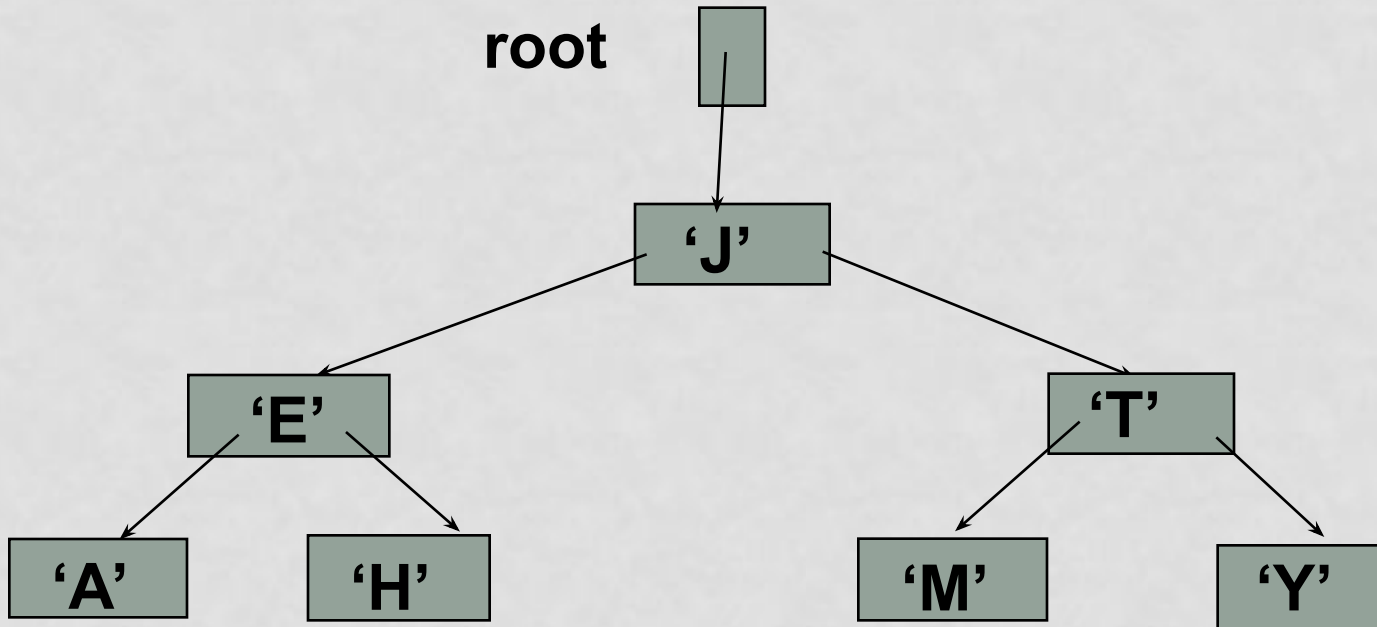


BINARY SEARCH TREE

- A particular form of binary tree, such that:
- For any node in the tree containing data value x :
 - The right subtree has data values $> x$
 - The left subtree has data values $< x$



EXAMPLE BINARY SEARCH TREE

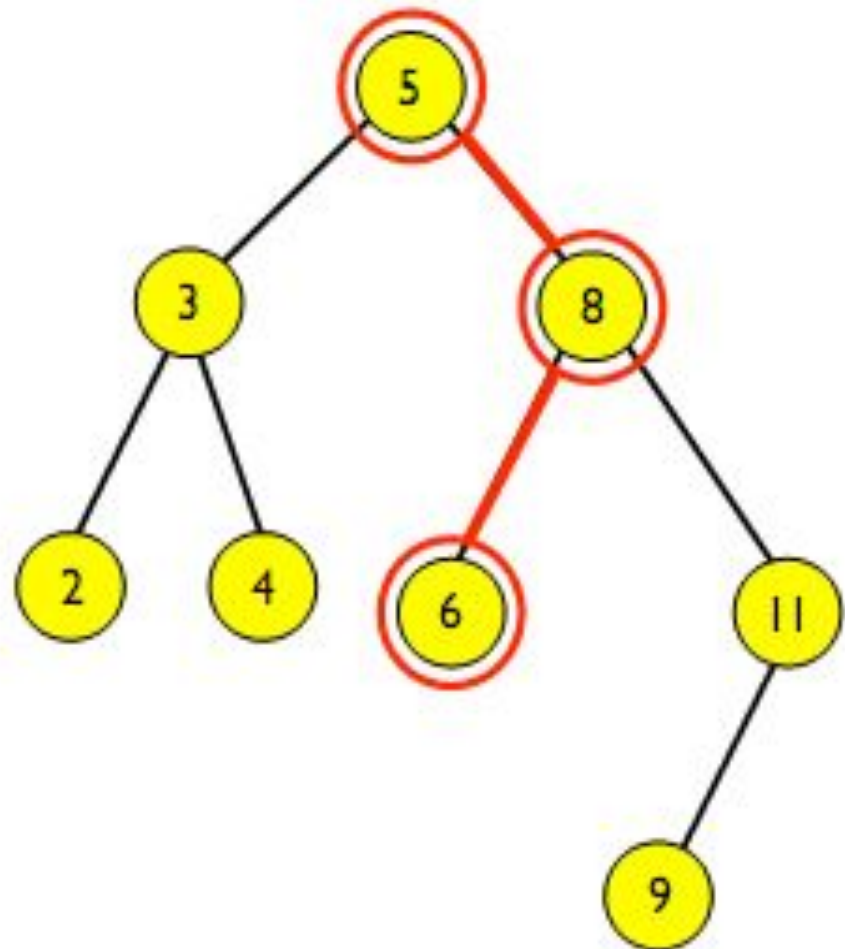


FINDING A NODE IN A BINARY SEARCH TREE

Find $k = 6$:

Is $k < 5$? No, go right

Is $k < 8$? Yes, go left



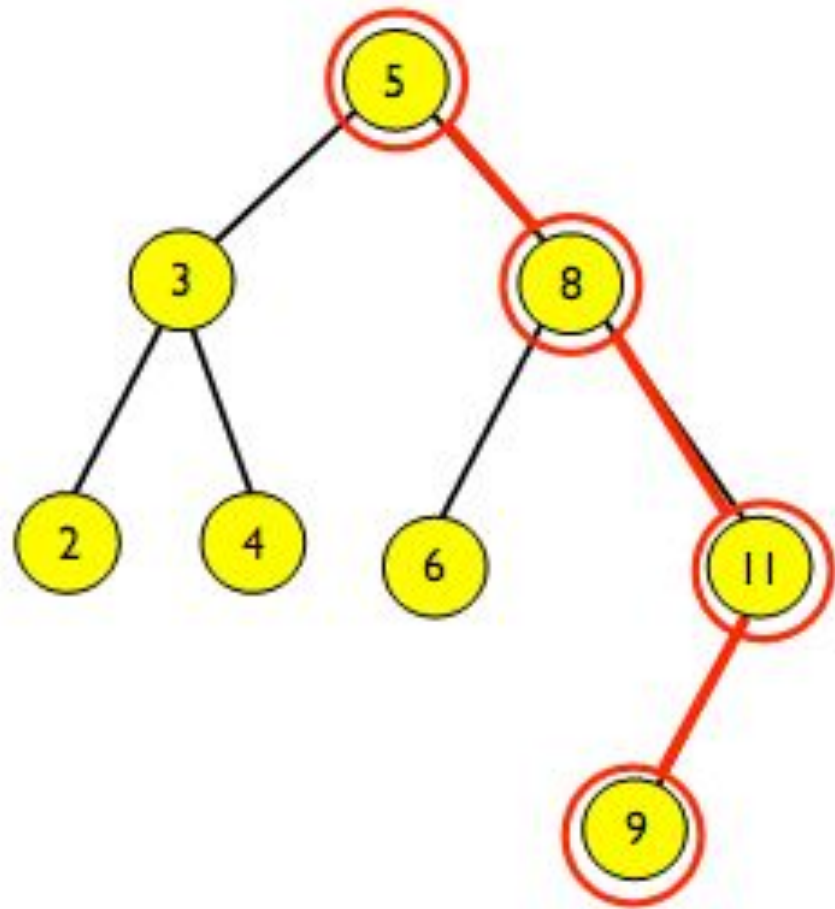
FINDING A NODE IN A BINARY SEARCH TREE

Find $k = 9$:

Is $k < 5$? No, go right

Is $k < 8$? No, go right

Is $k < 11$? Yes, go left



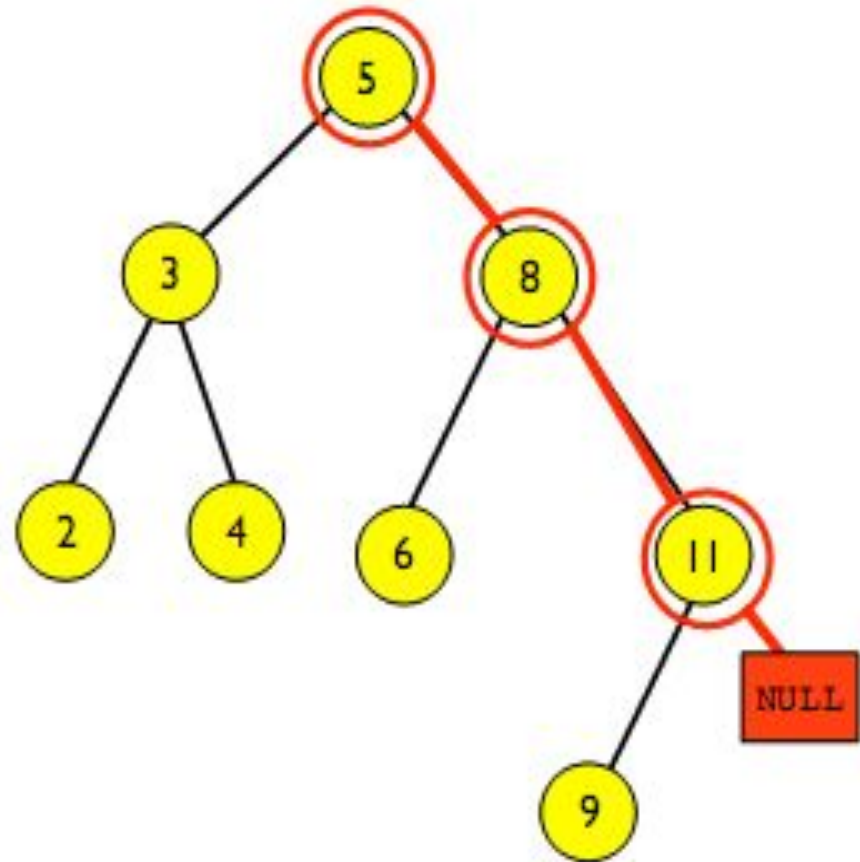
FINDING A NODE IN A BINARY SEARCH TREE

Find $k = 13$:

Is $k < 5$? No, go right

Is $k < 8$? No, go right

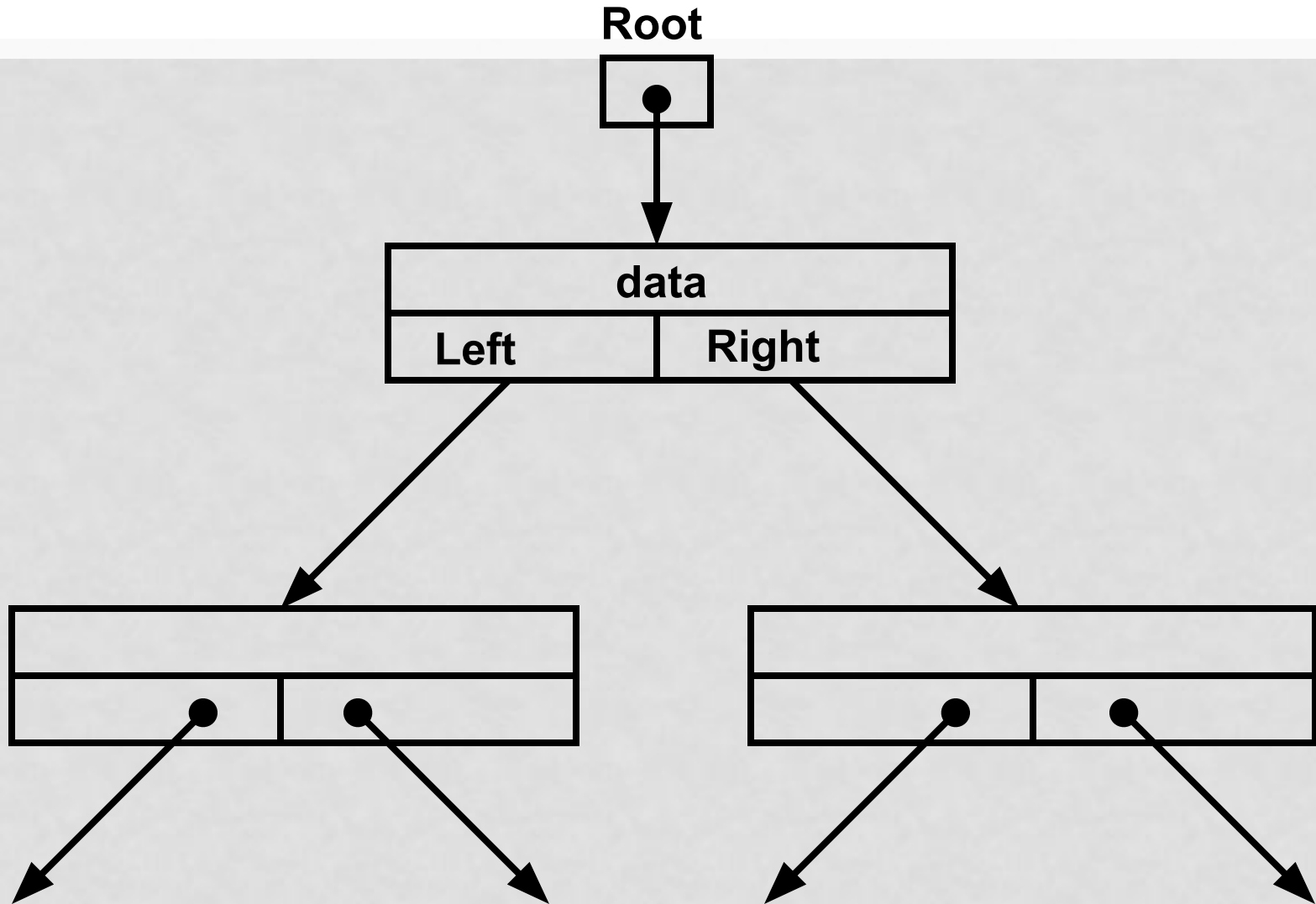
Is $k < 11$? No, go right



CONSTRUCTING A BST FROM A LIST

- 14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5
- Figure 5.1.6 from the book

POINTER-BASED IMPLEMENTATION OF A BINARY (SEARCH) TREE



Binary Search Tree

POINTER-BASED REPRESENTATION OF BINARY SEARCH TREE

```
struct Node          //node in the Tree
{
    int data;          //data portion
    Node * Left;       //pointer to left child
    Node * Right;      //pointer to right child
};
class BST
{
    public:
        Node * root;
        ...
};
```

BINARY SEARCH TREE: TRAVERSAL

```
void Inorder(Node * root)  
{  
if (root!=NULL) {  
    Inorder(root->Left);  
    cout>>root->data;  
    Inorder(root->Right);}  
}
```

```
void Preorder(Node * root)  
{  
    If (root!=NULL) {  
        cout>>root->data;  
        Preorder(root->Left);  
        Preorder(root->Right); }  
}
```

```
void Postorder(Node * root)  
{  
    If (root!=NULL) {  
        Postorder(root->Left);  
        Postorder(root->Right);  
        cout>>root->data;}  
}
```


INORDER TRAVERSE

```
Void traversein(node* r)
{
  If(r != NULL)
  {
    traversein(r->left);
    Cout<<r->data;
    traversein(r->right);
  }
}
BST t; t.traversein(t.getRoot());
```

READING

- Chapter 10, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.